

Protoportal Main — Admin & Backend Feature Review

Source of truth: local codebase at ~/Desktop/Projects/ /protoportal-main. Reviewed on 2026-05-10.

Executive summary

This project is a React + Vite wholesale portal backed by Supabase auth/database, with a lightweight admin panel embedded in the frontend for users whose `customers.role` is `admin`. The implemented admin scope is practical and fairly complete for catalogue operations: product CRUD, CSV import/export, customer approval/tiering, specials management, and order review/export. Backend logic is mostly client-to-Supabase with Row Level Security doing the protection, plus one serverless email endpoint for sending order PDFs.

Admin access model

- Admin UI is only surfaced when the logged-in customer profile has `role === 'admin'` (`src/components/Header.jsx`, `src/Root.jsx`).
- Normal approved customers land in the catalogue portal; unapproved customers see a pending approval screen.
- Supabase migration defines `public.is_admin()` and RLS policies so admin users can read/write all products, customers, and orders.

Main admin modules

Module	What it does	Implementation notes
Products	Add/edit products, archive/restore, search/filter, reorder sort positions, CSV import, CSV export, set “special” flags and visibility.	Implemented in <code>ProductsTab</code> in <code>src/pages/AdminPage.jsx</code> ; data access in <code>src/lib/products.js</code> .
Customers	Review all customers, approve accounts, change tier (regular/premium), import tiers from CSV, export customer list.	Implemented in <code>CustomersTab</code> ; data access in <code>src/lib/customers.js</code> .
Specials	Curate weekly specials from active products and choose visibility by customer tier.	Implemented in <code>SpecialsTab</code> ; depends on product <code>is_special</code> and <code>special_visibility</code> .
Orders	View recent orders with customer details and export them to CSV.	Implemented in <code>OrdersTab</code> ; admin fetch joins orders with customer name/email/tier.

Module details

1) Products module

- Manual product form supports code, name, price ex VAT, stock, min qty, image URL, category, case pack, lead time, trade note, new-stock flag, and archive flag.
- List view includes thumbnail, code, name, price, stock, category path, active/archived status, special status, special visibility, and row-level actions.
- Sorting can be adjusted manually by swapping `sort_order` values up/down.
- CSV import maps a simple sheet into product upserts; export emits product data back out as CSV.
- Products can be marked as “special” and limited to `all`, `premium`, or `regular` customers.

2) Customers module

- Shows all users with name, email, phone, delivery address, tier, approval state, registration date, and role label.
- Admins can approve/disapprove accounts by toggling `is_approved`.
- Admins can set tier manually to `regular` or `premium`.

- Bulk CSV tier import expects email, tier.
- Dashboard summary highlights pending approvals and tier counts.

3) Specials module

- Provides a dedicated merchandising view for weekly specials.
- Separates current specials from non-special active products.
- Allows quick add/remove and visibility targeting by customer segment.
- Frontend portal respects this visibility when users toggle specials view.

4) Orders module

- Shows customer name, email, tier, item count, order total ex VAT, status, and date.
- Supports CSV export of the order list.
- Read-only in current implementation: no admin status editing, fulfillment workflow, or internal notes UI.

Backend / data features

Supabase auth + profile bootstrap

- Auth supports login and signup from the portal UI.
- A database trigger auto-creates a customers row when a new auth user is created.
- New users require admin approval before they can access the portal.

Products table

- Stores pricing, image URL, stock, category path, tags/badges, archive state, new/special flags, segment visibility, sort order, minimum qty, and trade notes.
- RLS allows approved customers to read active products and admins to do everything.

Customers table

- Stores email, name, phone, delivery address, tier, role, approval status, and created date.
- Users can read/update their own profile; admins can read all and update/delete broadly.

Orders table

- Stores line items as JSON, total ex VAT, status, customer link, and created date.
- Users can create/read their own orders; admins can read/manage all via RLS.

Operational logic beyond the admin UI

- **Order persistence:** checkout writes orders to Supabase before sending email.
- **Auto-upgrade rule:** a regular customer is auto-promoted to premium when an order total exceeds R4,000 and at least one line quantity exceeds 10.
- **Email/PDF generation:** api/send-order.js builds a branded PDF with PDFKit and emails it via Resend.
- **Fallback catalogue:** if live product fetch fails, the storefront falls back to static JSON stock data.

Obvious caveats / risks

Area	Caveat	Impact
CSV parsing	CSV is parsed with naive split(', ') logic in the browser.	Quoted commas, escaped values, or complex CSVs may import incorrectly.
Bulk tier import	bulkSetTiersFromCsv updates customers one row at a time from the client.	Slow for large lists; partial failures are hard to detect cleanly.
Order admin workflow	Orders are view/export only in admin.	

Area	Caveat	Impact
		No shipment/status management, assignment, note-taking, or audit workflow yet.
Product uniqueness	Product upsert uses <code>onConflict: 'code'</code> , but the SQL migration shown does not define a unique constraint on <code>products.code</code> .	Upsert may fail or behave unexpectedly unless the live database has an extra unique index not shown here.
Supabase-only trust model	Most admin writes happen directly from the frontend to Supabase.	Security depends heavily on correct RLS and profile role integrity.
Email dependency	Order submission requires <code>RESEND_API_KEY</code> and reachable image URLs for richer PDFs.	Missing env vars or broken images degrade email/PDF output.
Status controls	Order status exists in schema but there is no admin UI to change it.	Operational teams may need direct DB access or a future admin enhancement.

Bottom line

The admin/backend implementation is real and usable, not just scaffolded. It covers the core catalogue and account-management needs for a wholesale portal, with a clean role-gated admin surface. The biggest gaps are around robustness (CSV handling, batched ops), richer order operations, and making sure the database schema fully matches the client assumptions.

Key files reviewed: `src/pages/AdminPage.jsx`, `src/Root.jsx`, `src/lib/products.js`, `src/lib/customers.js`, `src/lib/orders.js`, `api/send-order.js`, `migrations/001_initial.sql`, `src/components/Header.jsx`.